

BML Annotation Tables

Complete Reference: Interval Algebra, Manipulation,
Visualization, and Worked Examples

Brain Modulation Laboratory · MGH / University of Pittsburgh

April 2026

Contents

1	The Annotation Table: Core Data Structure	3
1.1	Rationale	3
1.2	The four mandatory columns	3
1.3	Creating annotation tables	3
2	The ROI Table and the (s1, t1, s2, t2) Coordinate System	4
2.1	What is an ROI table?	4
2.2	The linear coordinate map	4
2.3	Why sample midpoints?	4
3	Creating and Loading Annotation Tables	5
3.1	From scratch	5
3.2	From BIDS TSV files (bml_annot_read_tsv)	5
3.3	From FieldTrip event structs (bml_event2annot)	5
3.4	Scanning a folder to build the initial ROI table	6
4	bml_annot_extend — Temporal Dilation and Erosion	6
4.1	Rationale	6
5	bml_annot_filter / filterout — Selection by Time Window	7
5.1	Rationale	7
5.2	Overlap condition	7
6	bml_annot_intersect — Temporal Intersection	7
6.1	Rationale	7
7	bml_annot_union — Temporal Union	8
7.1	Rationale	8
8	bml_annot_difference — Set Subtraction	9
8.1	Rationale	9
9	bml_annot_consolidate — Merging by Criterion	9
9.1	Rationale	9

9.2 Algorithm	10
10 bml_annot_blocks — Run-Length Encoding and Transitions	10
10.1 Rationale	10
11 bml_annot_shadow — Filling the Gaps	11
11.1 Rationale	11
12 bml_annot_coverage — Measuring How Much is Covered	12
13 bml_annot_overlap — Detecting Conflicts	12
14 Joining Tables	13
14.1 bml_annot_left_join — Key-Based Join	13
14.2 bml_annot_transfer — Temporal Overlap Join	13
15 bml_annot_calculate — Feature Extraction over Epochs	13
16 bml_annot_detect — Threshold-Based Event Detection	14
17 bml_annot2raw / bml_raw2annot — Signal Encoding	14
17.1 Annotations to binary/count signals (bml_annot2raw)	14
18 bml_annot_plot — Visualization	15
19 bml_annot_to and bml_annot_rowbind — Utilities	15
19.1 bml_annot_to — Change Time Reference	15
19.2 bml_annot_rowbind — Vertical Concatenation	15
20 Complete Worked Example: From Raw Events to Clean Trial Features	16
21 Annotation Algebra: Complete Reference Table	17
A Interval Algebra: Mathematical Summary	19

1 The Annotation Table: Core Data Structure

1.1 Rationale

Every piece of timing information in BML — a stimulus onset, a file on disk, an artifact window, a sync interval, a spike — is stored in a single, uniform structure: the **annotation table**. This makes every function interoperable: a function that accepts an annotation table accepts *any* kind of time-stamped data.

The key insight is that everything interesting in neuroscience has a **start time** and an **end time**. Even instantaneous events (a spike, a button press) are stored with `ends = starts`, so the same code handles both.

1.2 The four mandatory columns

id	starts	ends	duration	extra cols...
1	10.000	10.500	0.500	'STIM_ON'
2	11.200	11.700	0.500	'STIM_ON'
3	13.500	14.200	0.700	'RESPONSE'
↓ auto-assigned integer	↓ seconds on master clock	↓ seconds on master clock	↓ ends—starts auto-computed	↓ arbitrary extra data

- **id**: unique integer per row. Auto-assigned in ascending order sorted by starts. Not stable across calls to `bml_annot_table` — never use `id` as a persistent key.
- **starts**: interval start time, in seconds on the master clock.
- **ends**: interval end time ($\geq$ starts). Instantaneous events: `ends = starts`.
- **duration**: `ends - starts`, always recomputed; never trust an existing duration column if `starts/ends` were modified.

1.3 Creating annotation tables

```

1 % 1. From a table with starts and ends columns
2 T = table([1.0; 2.5; 4.0], [1.5; 3.0; 4.8], ...
3           'VariableNames', {'starts', 'ends'});
4 T.type = {'STIM_ON'; 'STIM_ON'; 'RESPONSE'};
5 annot = bml_annot_table(T);
6 % -> id=[1;2;3], duration=[0.5;0.5;0.8] auto-computed
7
8 % 2. From a two-column numeric matrix [starts, ends]
9 annot = bml_annot_table([10 10.5; 11.2 11.7; 13.5 14.2]);
10
11 % 3. From a struct
12 s.starts = [10; 11.2; 13.5];
13 s.ends   = [10.5; 11.7; 14.2];
14 s.type   = {'A'; 'A'; 'B'};
15 annot = bml_annot_table(struct2table(s));

```

Listing 1: Three ways to call `bml_annot_table`

Warning

`bml_annot_table()` sorts rows by starts and re-numbers `id` from 1. If you add a row manually and call it again, all `ids` change. Use `id` only as a within-table reference, never as a cross-table key.

2 The ROI Table and the (s1, t1, s2, t2) Coordinate System

2.1 What is an ROI table?

A **Region of Interest (ROI) table** is an annotation table extended with file metadata and *sync coordinates*. Each row describes a time window within a specific file on disk.

Column	Type	Source	Meaning
id, starts, ends, duration	double	annotation	Time window (master clock)
folder	string	filesystem	Directory containing the file
name	string	filesystem	Filename
nSamples	int	file header	Total number of samples
Fs	double	file header	Nominal sampling rate (Hz)
filetype	string	config	e.g. 'trellis', 'zoom', 'neuroomega'
chantype	string	config	e.g. 'analog', 'digital', 'lfp'
s1	int	sync	First anchor: sample index
t1	double	sync	First anchor: master-clock time of sample s1
s2	int	sync	Second anchor: sample index
t2	double	sync	Second anchor: master-clock time of sample s2

2.2 The linear coordinate map

The four sync coordinates (s1, t1, s2, t2) define an affine map between sample index space and master-clock time:

Index-to-time conversion

$$F_{s,\text{eff}} = \frac{s_2 - s_1}{t_2 - t_1}, \quad t(i) = \frac{i - s_1}{F_{s,\text{eff}}} + t_1 - \frac{0.5}{F_{s,\text{eff}}} = \frac{i}{F_{s,\text{eff}}} - \frac{0.5}{F_{s,\text{eff}}} + \frac{s_2 t_1 - t_2 s_1}{s_2 - s_1}$$

Time-to-index conversion

$$i(t) = \text{round}\left(\frac{t_2 s_1 - s_2 t_1 + (s_2 - s_1)t}{t_2 - t_1}\right)$$

2.3 Why sample midpoints?

Sample i occupies the time window $[\frac{i-1}{F_s}, \frac{i}{F_s})$. Its *midpoint* is $\frac{i}{F_s} - \frac{0.5}{F_s}$. The coordinates t1 and t2 refer to the midpoints of samples s1 and s2, respectively.

This matters because: if you know the start of a 30 kHz file is t_{start} , then sample 1 starts at t_{start} but its midpoint (what t1 records) is $t_{\text{start}} + \frac{0.5}{30000}$. The default initialization is:

$$s_1 = 1, \quad t_1 = \text{starts} + \frac{0.5}{F_s}, \quad s_2 = N_{\text{samples}}, \quad t_2 = \text{ends} - \frac{0.5}{F_s}$$



```

1 % Build from file scan
2 roi = bml_roi_table(bml_info_raw(struct('folder', '/data/p001/s01')));
3

```

```

4 % Default coordinates:
5 % roi.s1 = 1 (first sample)
6 % roi.t1 = roi.starts + 0.5/roi.Fs (midpoint of sample 1)
7 % roi.s2 = roi.nSamples
8 % roi.t2 = roi.ends - 0.5/roi.Fs (midpoint of last sample)
9
10 % After synchronization, t1/t2 are updated to master-clock times.
11 % Convert between sample index and time:
12 t = bml_idx2time(roi(1,:), 15000); % time of sample 15000
13 i = bml_time2idx(roi(1,:), 52312.5); % sample nearest to 52312.5 s

```

Listing 2: Building and using an ROI table

3 Creating and Loading Annotation Tables

3.1 From scratch

```

1 % Minimal: just starts and ends
2 spikes = bml_annot_table(table([1.2;1.8;2.4],[1.2;1.8;2.4],...
3   'VariableNames',{'starts','ends'}));
4
5 % With extra columns
6 trials = table([10;20;30], [10.5;20.5;30.5], {'go';'nogo';'go'}, [1;2;3],...
7   'VariableNames',{'starts','ends','trial_type','value'});
8 trials = bml_annot_table(trials);

```

3.2 From BIDS TSV files (bml_annot_read_tsv)

BIDS event files use onset (seconds from task start) and duration columns. `bml_annot_read_tsv` renames onset→starts and computes ends = starts + duration.

```

1 events = bml_annot_read_tsv( ...
2   'sub-DM1001_ses-intraop_task-speech_events.tsv');
3 % Columns: id, starts, ends, duration, trial_type, response_time, ...
4 % Also extracts sub/ses/task from filename if AppendColsFromFilename=true
5
6 % Write back after sync:
7 bml_annot_write_tsv(events_synced, 'sub-DM1001_...-sync.tsv');
8 % Renames starts→onset, drops id, writes tab-delimited

```

3.3 From FieldTrip event structs (bml_event2annot)

```

1 % Read Ripple (Trellis) events
2 ft_events = ft_read_event(fullfile(roi.folder{1},roi.name{1}));
3
4 cfg = [];
5 cfg.roi = roi(strcmp(roi.filetype,'trellis'),:);
6 cfg.Fs = 30000;
7 annot_events = bml_event2annot(cfg, ft_events);
8 % Columns: id, starts, ends, duration, type, value, sample
9 % sample = raw sample index; starts/ends = master-clock seconds via roi
10
11 % Quick shortcut used throughout BML:
12 master_events = bml_event2annot([], bml_read_event(trellis_roi));

```

3.4 Scanning a folder to build the initial ROI table

```

1 % Step 1: scan folder, infer file type from extension/name pattern
2 info = bml_info_raw(struct('folder', '/data/p001/s01', ...
3                           'filetype', {'trellis','zoom','neuroomega'}));
4
5 % Step 2: create ROI table with default sync coordinates
6 roi = bml_roi_table(info);
7
8 % roi now has one row per file, with:
9 %   s1=1, t1=starts+0.5/Fs (OS timestamps, unreliable)
10 %   s2=nSamples, t2=ends-0.5/Fs
11 % Sync functions update t1/t2 to master-clock times.

```

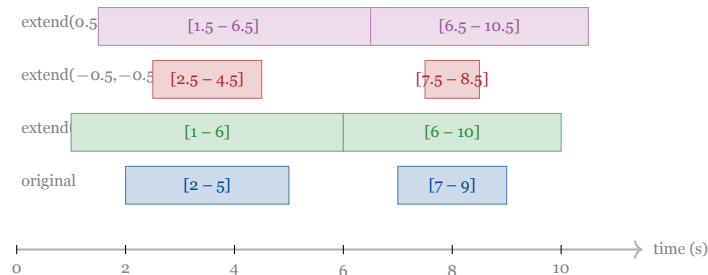
4 bml_annot_extend — Temporal Dilation and Erosion

4.1 Rationale

Many analysis steps need a *window around* an event rather than the event itself. `bml_annot_extend` adds (or subtracts) time on either side of every row, without touching other columns.

Extension rule

$\text{starts}_{\text{new}} = \text{starts} - \text{ext1}$, $\text{ends}_{\text{new}} = \text{ends} + \text{ext2}$
 Positive = expand outward. Negative = shrink inward (erosion).



```

1 % Add 0.5 s context on each side of every trial
2 trial_windows = bml_annot_extend(trials, 0.5);
3
4 % Pre-stimulus baseline: 0.5 s ending at stimulus onset
5 baseline = bml_annot_extend(stim_events, 0.5, 0);
6 % Now baseline.ends == stim_events.starts, baseline.starts == stim_events.starts -
   0.5
7
8 % Shrink (erode) artifact windows by 50 ms to be conservative
9 artifacts_trimmed = bml_annot_extend(artifacts, -0.05);
10
11 % Asymmetric: 1 s before onset, 2 s after
12 analysis_win = bml_annot_extend(stim_events, 1.0, 2.0);
13
14 % Sync coarsetol: expand the ROI window to capture master events
15 % that might fall ±100 s outside the file OS timestamps
16 roi_expanded = bml_annot_extend(roi_file, 100);

```

5 bml_annot_filter / filterout — Selection by Time Window

5.1 Rationale

Often you have a table of events and a table of windows, and you want to keep only the events that fall inside the windows (or remove those that overlap bad periods). This is the most common single operation in a BML pipeline.

5.2 Overlap condition

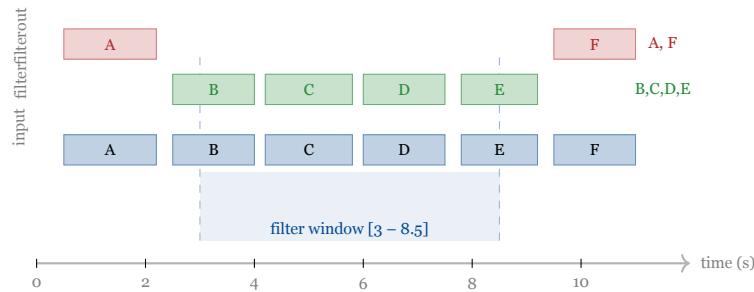
Two intervals $[a_1, a_2]$ and $[b_1, b_2]$ overlap if and only if:

Overlap condition

$$a_1 < b_2 \text{ AND } a_2 > b_1$$

(They overlap when neither is entirely before the other)

With `cfg.overlap = f`: additionally require $|a \cap b|/|a| \geq f$.



Key point: `bml_annot_filter` keeps the rows of `annot` intact (start/end times are not clipped). It is a *selection* operation, not a clip. Use `bml_annot_intersect` if you also want to clip to the filter boundary.

```

1 % Keep only trials within the session window
2 session = bml_annot_table(table(52300, 52600, 'VariableNames', {'starts', 'ends'}));
3 trials_in_session = bml_annot_filter(trials, session);
4
5 % Remove trials that overlap any artifact
6 clean_trials = bml_annot_filterout(trials, artifacts);
7
8 % Keep only trials where >= 50% of the trial is inside the analysis window
9 cfg = []; cfg.overlap = 0.5;
10 mostly_inside = bml_annot_filter(cfg, trials, analysis_window);
11
12 % Restrict master events to ±100s of a specific file (sync coarsetol pattern)
13 file_window = bml_annot_extend(roi(3,:), 100); % expand file ROI by 100s
14 local_events = bml_annot_filter(master_events, file_window);

```

6 bml_annot_intersect — Temporal Intersection

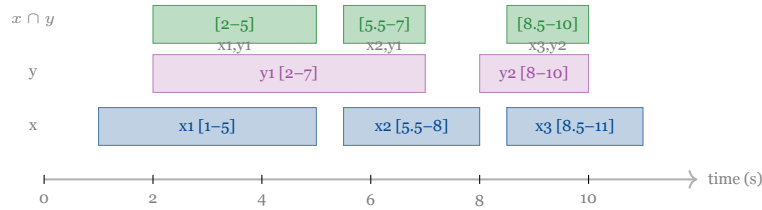
6.1 Rationale

`bml_annot_filter` selects whole rows; `bml_annot_intersect` *clips* them to the overlap region. The result intervals are shorter than either input. Crucially, it records where each output interval came from (`x_id`, `y_id`).

Intersection of two intervals

$$[a_1, a_2] \cap [b_1, b_2] = [\max(a_1, b_1), \min(a_2, b_2)]$$

Valid (non-empty) only when $\max(a_1, b_1) < \min(a_2, b_2)$.



Note: x2 overlaps y1 only partially ([5.5-7], not [5.5-8]). x1 and x3 have no intersection with y2; x3 overlaps y2 at [8.5-10], not [8.5-11].

```

1 % Basic intersection: clips trials to session windows
2 clipped = bml_annot_intersect(trials, session_windows);
3 % clipped has x_id (trial id) and y_id (session id) columns
4
5 % Keep columns from both tables (cfg.keep = 'both', default)
6 cfg = []; cfg.keep = 'both';
7 overlap_info = bml_annot_intersect(cfg, trials, artifacts);
8 % overlap_info.trials_id, overlap_info.artifacts_id, plus all extra columns
9
10 % Keep only x columns (strip y metadata from output)
11 cfg = []; cfg.keep = 'x';
12 clipped_trials = bml_annot_intersect(cfg, trials, session_windows);
13
14 % Per-channel intersection (each channel has its own artifact list)
15 cfg = []; cfg.groupby = 'channel';
16 per_ch = bml_annot_intersect(cfg, spikes, artifacts_per_channel);

```

7 bml_annot_union — Temporal Union

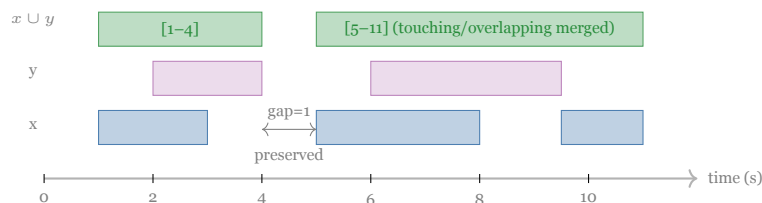
7.1 Rationale

Merge overlapping or touching intervals into the fewest possible non-overlapping intervals. Used to combine artifact lists from multiple sources, merge session windows, etc.

Merging condition

Intervals $[a_1, a_2]$ and $[b_1, b_2]$ (with $b_1 \geq a_1$) are merged when:

$b_1 \leq a_2 + \text{timetol}$ (they touch or overlap within tolerance)



```

1 % Merge artifact lists from two annotators
2 all_artifacts = bml_annot_union(artifacts_A, artifacts_B);
3
4 % Self-union: consolidate a single table's overlaps

```



```

5 % (useful if you added rows that now overlap)
6 clean = bml_annot_union(dirty_annot);
7
8 % Wider tolerance: treat gaps < 10 ms as touching
9 cfg = []; cfg.timetol = 0.01;
10 merged = bml_annot_union(cfg, artifacts_A, artifacts_B);

```

8 bml_annot_difference — Set Subtraction

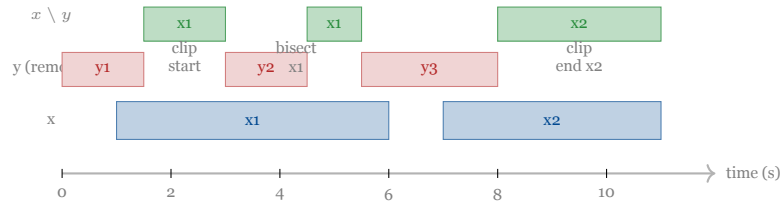
8.1 Rationale

Remove regions covered by y from x . The result may split x intervals into pieces. Original x column data is preserved on each fragment.

Set difference

$x \setminus y$: all parts of x that do not overlap y .

If y bisects an x interval: the result has two fragments from the original row.



Five geometric cases handled internally:

1. y fully covers x $\square$ x row removed entirely
2. y clips the start of x $\square$ fragment $[y.ends, x.ends]$
3. y clips the end of x $\square$ fragment $[x.starts, y.starts]$
4. y bisects x $\square$ two fragments: $[x.starts, y.starts]$ and $[y.ends, x.ends]$
5. y is outside x $\square$ x row kept as-is

```

1 % Remove artifact periods from trial windows
2 clean_trials = bml_annot_difference(trial_windows, artifacts);
3 % Trials are split where artifacts bisect them.
4 % clean_trials.x_id_ links each fragment back to its original trial.
5
6 % Per-channel artifact removal (artifacts labelled by channel)
7 cfg = []; cfg.groupby_x = 'channel'; cfg.groupby_y = 'channel';
8 clean_per_ch = bml_annot_difference(cfg, lfp_windows, chan_artifacts);
9
10 % What remains after removing all high-gamma bursts from the session?
11 session_gaps = bml_annot_difference(session_window, hg_bursts);

```

9 bml_annot_consolidate — Merging by Criterion

9.1 Rationale

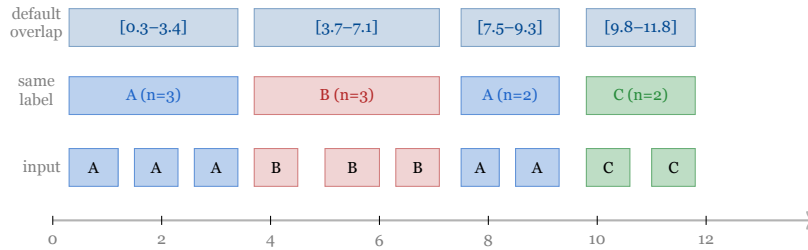
Given a sorted annotation table, consolidate (merge) consecutive rows that satisfy a custom criterion. This is the most flexible grouping primitive in BML: it handles overlap merging, same-label grouping, fixed-size batching, and custom condition merging.

9.2 Algorithm

Starting from row 1, greedily try to add each next row to the current group S :

1. Tentatively add row r to S
2. Evaluate $\text{criterion}(S)$
3. If true: keep r in S , continue to next row
4. If false: close current group, start new group with r

The default criterion is overlap/contiguity: $\text{criterion}(S) = S.\text{starts}(\text{end}) \leq \max(S.\text{ends}(1:\text{end}-1))$.



```

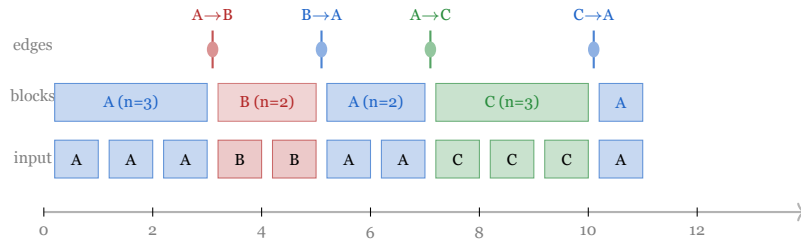
1 % 1. Default criterion: merge overlapping/touching (same as bml_annot_union)
2 cons = bml_annot_consolidate(annot);
3
4 % 2. Same-label grouping (like run-length encoding)
5 cfg = []; cfg.criterion = @(x) length(unique(x.label)) == 1;
6 cons_by_label = bml_annot_consolidate(cfg, annot);
7
8 % 3. Fixed group size (batch trials in groups of 4)
9 cfg = []; cfg.criterion = @(x) height(x) <= 4;
10 batches = bml_annot_consolidate(cfg, trials);
11
12 % 4. Contiguous files with same depth (NeuroOmega sync pattern)
13 cfg = [];
14 cfg.criterion = @(x) (length(unique(x.depth))==1) && ...
15     (abs((max(x.ends)-min(x.starts)) - sum(x.duration)) < 1e-2);
16 cons_depth = bml_annot_consolidate(cfg, neuro_info);
17
18 % Output columns always include:
19 %   cons_duration = sum(duration of merged rows)
20 %   id_starts     = id of first row in group
21 %   id_ends       = id of last row in group
22 %   cons_n        = number of rows merged

```

10 bml_annot_blocks — Run-Length Encoding and Transitions

10.1 Rationale

Given a sequence of annotations where each row has a label (e.g., a stimulus category, a behavioral state, a coding label), `bml_annot_blocks` groups consecutive rows with the *same label* into blocks and records the *transitions* between blocks. This is run-length encoding applied to time-series annotations.



```

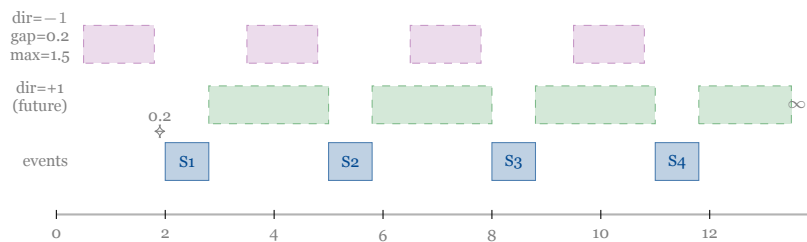
1 % Detect blocks of same stimulus condition
2 cfg = []; cfg.label = 'trial_type';
3 [blocks, edges] = bml_annot_blocks(cfg, trial_annot);
4 % blocks: id, starts, ends, duration, label, cons_n (how many rows merged)
5 % edges: id, starts, ends, pre_label, post_label
6
7 % With win_edges: create a window around each transition
8 cfg = []; cfg.label = 'depth'; cfg.win_edges = 2.0; % ±2s window
9 [blocks, edges] = bml_annot_blocks(cfg, recording_info);
10 % edges.starts = transition_time - 2.0
11 % edges.ends   = transition_time + 2.0
12
13 % Group by an additional variable (e.g. per session)
14 cfg = []; cfg.label = 'state'; cfg.group_by = 'session_id';
15 [blocks, edges] = bml_annot_blocks(cfg, behavioral_coding);

```

11 bml_annot_shadow — Filling the Gaps

11.1 Rationale

Given a set of events, generate annotations that occupy the *space between* them. The primary use case is creating pre-stimulus baselines: for each stimulus, get the silence that precedes it.



```

1 % Pre-stimulus baseline: window before each stimulus (gap 0, max 1.5s)
2 cfg = [];
3 cfg.direction = -1; % look into the past
4 cfg.gap_duration = 0; % shadow touches the event
5 cfg.max_duration = 1.5; % cap at 1.5 s
6 baseline = bml_annot_shadow(cfg, stim_events);
7 % baseline(i).ends == stim_events(i).starts (gap=0)
8 % baseline(i).starts == stim_events(i).starts - min(1.5, gap_to_prev_event)
9
10 % Post-response silence (future direction, with 200 ms gap)
11 cfg = []; cfg.direction = 1; cfg.gap_duration = 0.2; cfg.max_duration = 2.0;
12 post_resp = bml_annot_shadow(cfg, response_events);
13
14 % The last shadow always extends to ±inf (clipped by max_duration if set).
15 % Use bml_annot_filter afterward to restrict to a session window:
16 post_resp = bml_annot_filter(post_resp, session_window);

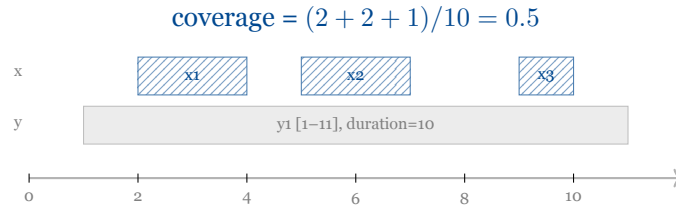
```

12 bml_annot_coverage — Measuring How Much is Covered

Coverage formula

$$\text{coverage}(y_j) = \frac{\sum_i |x_i \cap y_j|}{|y_j|}$$

Returns a value in $[0, 1]$: fraction of y_j 's duration covered by x .



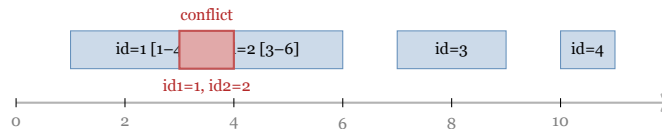
```

1 % Fraction of each trial covered by artifacts
2 cov = bml_annot_coverage(artifacts, trials);
3 % cov has same columns as trials plus 'coverage' (0 to 1)
4
5 % Keep only trials with < 10% artifact coverage
6 clean_trials = trials(cov.coverage < 0.1, :);
7
8 % Per-channel: artifact coverage per trial per channel
9 cfg = []; cfg.groupby_x = 'channel';
10 cov_per_ch = bml_annot_coverage(cfg, chan_artifacts, trials);
11 % One row per (trial x channel) combination
12
13 % Use custom column name
14 cfg = []; cfg.colname = 'artifact_fraction';
15 cov = bml_annot_coverage(cfg, artifacts, trials);

```

13 bml_annot_overlap — Detecting Conflicts

`bml_annot_overlap` finds all pairs of rows within the *same* table that overlap each other. The result lists the overlapping pair IDs and the overlap region. Used for validation and debugging.



```

1 % Validate that a table has no overlaps (required before passing as y to intersect)
2 ovlp = bml_annot_overlap(annot);
3 assert isempty(ovlp), 'Table has %d overlapping pairs', height(ovlp);
4
5 % Find and inspect overlapping pairs
6 ovlp = bml_annot_overlap(my_events);
7 if ~isempty(ovlp)
8     disp(ovlp); % id1, id2, starts, ends of each overlap
9     % Fix: resolve by merging or trimming conflicting rows
10    my_events = bml_annot_union(my_events); % merge overlapping pairs
11 end

```

14 Joining Tables

14.1 bml_annot_left_join — Key-Based Join

Standard relational left join: all rows of the left table are preserved; columns from the right table are added where the key matches.

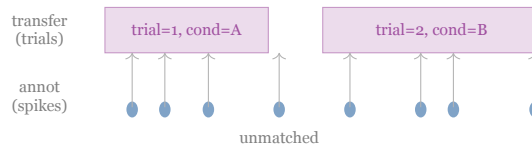
```

1 % Add subject/session metadata to each trial
2 cfg = []; cfg.keys = {'session_id'};
3 trials_with_meta = bml_annot_left_join(cfg, trials, session_metadata);
4
5 % Add per-electrode impedance to spike annotations
6 cfg = []; cfg.keys = {'channel'};
7 spikes_full = bml_annot_left_join(cfg, spikes, electrode_info);

```

14.2 bml_annot_transfer — Temporal Overlap Join

For each row in annot, find which row(s) in transfer contain it (by time overlap), and copy selected columns. Useful for assigning epoch-level labels to individual events within the epoch.



```

1 % Assign trial condition label to each spike
2 cfg = []; cfg.select = {'trial_id', 'condition', 'response'};
3 spikes_labeled = bml_annot_transfer(cfg, spikes, trial_epochs);
4 % Each spike gets the trial_id/condition/response of the trial containing it.
5 % Spikes not inside any trial get NaN/empty for transferred columns.
6
7 % cfg.overlap: require spike to be at least 50% inside the epoch
8 cfg.overlap = 0.5;

```

15 bml_annot_calculate — Feature Extraction over Epochs

Applies scalar functions to raw signal data over each epoch defined by an annotation table. Each function receives a FT_DATATYPE_RAW segment and must return a scalar (or one scalar per channel).

```

1 % Define feature functions
2 rms_fn = @(raw) sqrt(mean(raw.trial{1}(1,:).^2));
3 peak2pk = @(raw) max(raw.trial{1}(1,:)) - min(raw.trial{1}(1,:));
4 log_pwr = @(raw) 10*log10(mean(raw.trial{1}(1,:).^2));
5
6 % Compute features for each trial
7 cfg = [];
8 cfg.roi = sync_roi(strcmp(sync_roi.filetype, 'trellis'),:);
9 cfg.epoch = trial_windows;
10 trial_features = bml_annot_calculate(cfg, trial_windows, ...
11 'rms', rms_fn, 'peak2peak', peak2pk, 'log_power', log_pwr);
12 % trial_features has all original columns plus rms, peak2peak, log_power
13
14 % bml_annot_describe: summary statistics per group
15 cfg = []; cfg.groupby = 'condition';

```

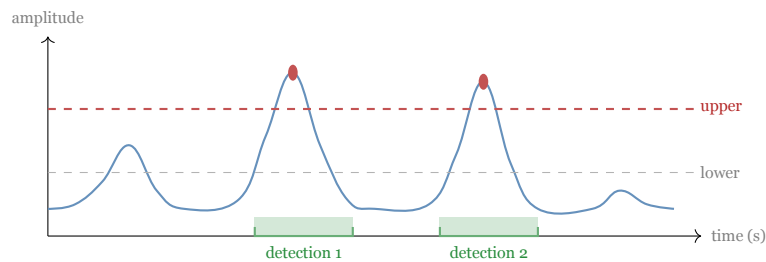
```

16 stats = bml_annot_describe(cfg, trial_features);
17 % stats: condition, variable, mean, median, std, min, max, count

```

16 bml_annot_detect — Threshold-Based Event Detection

Detects epochs where a signal exceeds a threshold. The algorithm finds peaks above the upper threshold, then expands each detection to the nearest zero-crossing of the lower threshold.



```

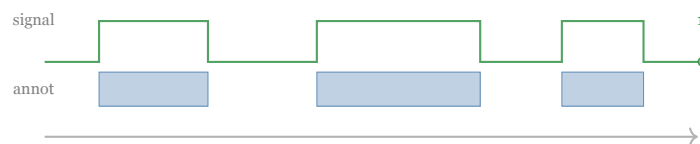
1 % Detect high-gamma (HG) bursts above 3 std
2 hg_raw = bml_load_continuous(struct('roi', hg_roi));
3
4 cfg = [];
5 cfg.threshold = [1.0, 3.0]; % [lower, upper] in units of the signal
6 cfg.channel = 'G14';
7 cfg.max_annots = 500;
8 hg_bursts = bml_annot_detect(cfg, hg_raw);
9 % hg_bursts: starts, ends, env_max, trial, label
10
11 % Single threshold (upper only; lower = 0 crossing)
12 cfg = []; cfg.threshold = 3.0;
13 detections = bml_annot_detect(cfg, envelope_raw);

```

17 bml_annot2raw / bml_raw2annot — Signal Encoding

17.1 Annotations to binary/count signals (bml_annot2raw)

Converts an annotation table to a continuous FT_DATATYPE_RAW signal: 1 where a row covers that sample, 0 elsewhere. With `cfg.count=true`, counts overlapping annotations per sample.



```

1 % Create binary stimulus presence signal (for cross-correlation with neural data)
2 cfg = [];
3 cfg.roi = trellis_roi;
4 cfg.label = 'stim_channel';
5 stim_raw = bml_annot2raw(cfg, stim_annotations);
6
7 % Count overlapping artifacts per sample (for quality metric)
8 cfg = []; cfg.count = true;
9 artifact_count_raw = bml_annot2raw(cfg, artifacts, trellis_roi);
10 % Each sample = number of simultaneously overlapping artifact annotations

```

```

11
12 % Per-label channels: one channel per unique label value
13 cfg = []; cfg.label_colname = 'trial_type';
14 coding_raw = bml_annot2raw(cfg, trial_annotations, roi);
15 % coding_raw has one channel per trial_type
16
17 % Inverse: extract time bounds from a raw struct back to annotation metadata
18 annot_meta = bml_raw2annot([], lfp_raw);
19 % annot_meta: starts, ends, trial, s1, t1, s2, t2, Fs, nSamples

```

18 bml_annot_plot — Visualization

Plots annotation intervals as horizontal line segments, with optional faceting (subplots) and y-axis grouping.

```

1 % Basic: one line per annotation id
2 bml_annot_plot([], trial_windows);
3
4 % Group by trial_type on y-axis
5 cfg = []; cfg.y = 'trial_type';
6 bml_annot_plot(cfg, trial_windows);
7
8 % Facet by session (one subplot per session)
9 cfg = [];
10 cfg.y = 'channel'; % y-axis = channel number
11 cfg.facet = 'session_id'; % one panel per session
12 bml_annot_plot(cfg, lfp_epochs);
13
14 % Use in a pipeline to inspect intermediate results
15 bml_annot_plot([], bml_annot_filter(events, session));

```

19 bml_annot_to and bml_annot_rowbind — Utilities

19.1 bml_annot_to — Change Time Reference

Shifts all times by a constant. Used when you need times relative to an event (e.g., stimulus onset = 0).

Time shift

$$\text{starts}_{\text{new}} = \text{starts} - t_0, \quad \text{ends}_{\text{new}} = \text{ends} - t_0$$

```

1 % Set stimulus onset as t=0
2 cfg = []; cfg.t0 = stim_events.starts(trial_idx);
3 relative_events = bml_annot_t0(cfg, all_events);
4
5 % Use roi.starts as reference (common in per-file analysis)
6 cfg = []; cfg.roi = roi(file_idx,:);
7 relative = bml_annot_t0(cfg, events_in_file);

```

19.2 bml_annot_rowbind — Vertical Concatenation

Concatenates multiple annotation tables, conforming all to the schema of the first non-empty table (fills missing columns with appropriate empty values).

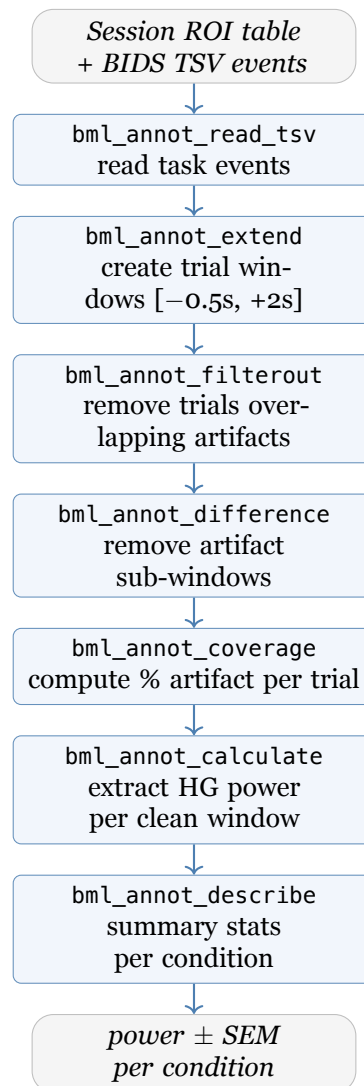
```

1 % Combine sync results from three methods
2 all_sync = bml_annot_rowbind(sync_roi_analog, sync_roi_no, sync_roi_ptb);
3
4 % Safer than [A; B; C] because it handles schema mismatches gracefully
5 % (missing columns are filled with NaN/empty)

```

20 Complete Worked Example: From Raw Events to Clean Trial Features

This example walks through a complete analysis pipeline using all the major annotation operations. Goal: extract high-gamma power from clean trial windows, per condition.



```

1 %% STEP 1: Load session ROI and build sync
2 roi = bml_roi_table(bml_info_raw(struct('folder', PATH_DATA)));
3 sync_roi = load(fullfile(PATH_SYNC, 'sync_roi_final.mat')); % pre-computed
4
5 %% STEP 2: Read task events (BIDS TSV)
6 events = bml_annot_read_tsv( ...
7     'sub-DM1001_ses-intraop_task-speech_events.tsv');
8 % events.starts = master-clock time (after sync applied at recording)
9 % events.trial_type = {'go', 'nogo', 'go', ...}

```



```

10
11 %% STEP 3: Create trial analysis windows
12 % Each trial = 0.5s before onset to 2s after
13 trial_windows = bml_annot_extend(events, 0.5, 2.0);
14 fprintf('Total trials: %d\n', height(trial_windows));
15
16 %% STEP 4: Load artifact annotations
17 artifacts = bml_annot_read_tsv('artifacts.tsv');
18 artifacts = bml_annot_filter(artifacts, session_window); % keep session-range only
19
20 %% STEP 5: Remove trials that are >50% artifact
21 cfg_filt = []; cfg_filt.overlap = 0.5;
22 clean_trials = bml_annot_filterout(cfg_filt, trial_windows, artifacts);
23 fprintf('Trials after artifact rejection: %d / %d\n', ...
24     height(clean_trials), height(trial_windows));
25
26 %% STEP 6: Compute per-trial artifact coverage
27 cov = bml_annot_coverage(artifacts, clean_trials);
28 clean_trials.artifact_pct = cov.coverage * 100;
29
30 %% STEP 7: Get clean (non-artifact) sub-windows for each trial
31 clean_windows = bml_annot_difference(clean_trials, artifacts);
32 % clean_windows.x_id_ links back to original trial id
33
34 %% STEP 8: Transfer trial metadata to clean windows
35 cfg_tr = []; cfg_tr.select = {'trial_type', 'value', 'session_id'};
36 clean_windows = bml_annot_transfer(cfg_tr, clean_windows, clean_trials);
37
38 %% STEP 9: Compute high-gamma power in each clean window
39 hg_roi = sync_roi(strcmp(sync_roi.filetype, 'trellis') & ...
40     strcmp(sync_roi.chantype, 'hg'), :);
41
42 log_pwr = @(raw) 10*log10(mean(raw.trial{1}(1,:).^2) + eps);
43
44 cfg_calc = []; cfg_calc.roi = hg_roi;
45 results = bml_annot_calculate(cfg_calc, clean_windows, ...
46     'log_power_hg', log_pwr);
47
48 %% STEP 10: Descriptive stats per condition
49 cfg_desc = []; cfg_desc.groupby = 'trial_type';
50 stats = bml_annot_describe(cfg_desc, results);
51 disp(stats); % trial_type | variable | mean | std | count | ...
52
53 %% STEP 11: Visualize
54 cfg_plot = []; cfg_plot.y = 'trial_type'; cfg_plot.facet = 'session_id';
55 bml_annot_plot(cfg_plot, clean_trials);
56 title('Clean trials by condition');

```

Listing 3: Complete pipeline: raw events to trial features

21 Annotation Algebra: Complete Reference Table

Function	Operation	What it does	Typical use
<code>bml_annot_table</code>	Constructor	Enforce schema, sort by starts, recompute duration	Create any annotation table
<code>bml_roi_table</code>	Constructor	Extend with file metadata + (s1,t1,s2,t2)	Build file/session index
<code>bml_annot_read_tsv</code>	I/O	Load BIDS TSV event files	Load task event logs
<code>bml_annot_write_tsv</code>	I/O	Save as BIDS TSV	Export synced events
<code>bml_event2annot</code>	Conversion	FieldTrip events → annotation table	Convert Ripple events
<code>bml_annot2raw</code>	Encoding	Annotations → binary/count signal	Cross-correlation, QC
<code>bml_raw2annot</code>	Encoding	Raw struct → time metadata	Extract file coordinates
<code>bml_annot_extend</code>	Dilation/Erosion	Expand or shrink interval boundaries	Analysis windows, coarsetol
<code>bml_annot_filter</code>	Selection	Keep rows that overlap a window	Restrict to session/ROI
<code>bml_annot_filterout</code>	Exclusion	Remove rows that overlap a window	Artifact rejection
<code>bml_annot_intersect</code>	Intersection	Clip to overlap region, track parent IDs	Find overlapping epochs
<code>bml_annot_union</code>	Union	Merge overlapping/touching intervals	Combine artifact lists
<code>bml_annot_difference</code>	Subtraction	Remove y from x , split intervals	Clean trial windows
<code>bml_annot Consolidate</code>	Grouping	Merge by custom criterion function	Run-length, batch, overlap
<code>bml_annot_blocks</code>	RLE	Contiguous same-label runs + transitions	Behavioral state segmentation
<code>bml_annot_shadow</code>	Gap filling	Annotations in the gaps between events	Pre/post-stimulus baseline
<code>bml_annot_coverage</code>	Measurement	Fraction of y covered by x	Artifact quality metric
<code>bml_annot_overlap</code>	Validation	Find conflicting (overlapping) pairs	Schema validation
<code>bml_annot_left_join</code>	Join	Key-based relational join	Add metadata columns
<code>bml_annot_transfer</code>	Temporal join	Copy columns based on time overlap	Assign trial labels to spikes
<code>bml_annot_calculate</code>	Aggregation	Apply functions to raw data per epoch	Feature extraction
<code>bml_annot_describe</code>	Statistics	Descriptive stats per group	Summarize features
<code>bml_annot_detect</code>	Detection	Threshold-crossing event detection	Detect HG bursts, peaks
<code>bml_annot_match</code>	Template match	Cross-correlation peak detection	Find pattern occurrences
<code>bml_annot_t0</code>	Translation	Shift all times by a constant	Event-relative time axis
<code>bml_annot_rowbind</code>	Concatenation	Vertical concat with schema conforming	Merge tables from loops
<code>bml_annot_conform_to</code>	Schema	Align column schema to template	Pre-concatenation fix

Function	Operation	What it does	Typical use
<code>bml_annot_rename</code>	Projection	Rename columns	Column name cleanup
<code>bml_annot_sample</code>	Sampling	Random row selection	Subsampling for tests
<code>bml_annot_plot</code>	Visualization	Horizontal bar plot, faceted	Inspect epoch tables

A Interval Algebra: Mathematical Summary

Notation

$$A = [a_1, a_2], B = [b_1, b_2], |A| = a_2 - a_1.$$

Overlap condition

$$A \cap B \neq \emptyset \Leftrightarrow a_1 < b_2 \text{ and } a_2 > b_1$$

Intersection

$$A \cap B = [\max(a_1, b_1), \min(a_2, b_2)], \text{ valid when non-empty.}$$

Union (two intervals)

$$A \cup B = [\min(a_1, b_1), \max(a_2, b_2)] \text{ when they overlap or touch } (b_1 \leq a_2 + \varepsilon).$$

Difference

$A \setminus B$: the part of A not covered by B . May produce 0, 1, or 2 intervals.

Extension/Erosion

$\text{extend}(A, e_1, e_2) = [a_1 - e_1, a_2 + e_2]$. Positive = dilate; negative = erode.

Coverage

$$\text{cov}(X, y) = \frac{\sum_i |X_i \cap y|}{|y|}, \text{ where } X = \{X_i\} \text{ is a set of intervals.}$$

Index $\leftrightarrow$ time (ROI coordinates)

$$F_{s,\text{eff}} = \frac{s_2 - s_1}{t_2 - t_1}, \quad t(i) = \frac{i}{F_{s,\text{eff}}} - \frac{0.5}{F_{s,\text{eff}}} + \frac{s_2 t_1 - t_2 s_1}{s_2 - s_1}, \quad i(t) = \text{round}\left(\frac{t_2 s_1 - s_2 t_1 + (s_2 - s_1)t}{t_2 - t_1}\right)$$